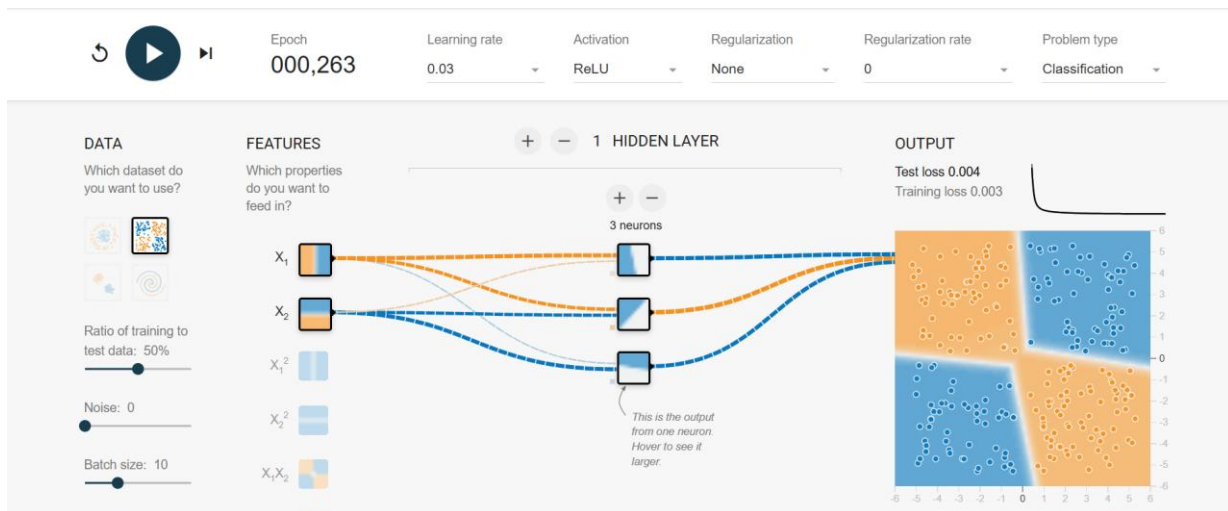


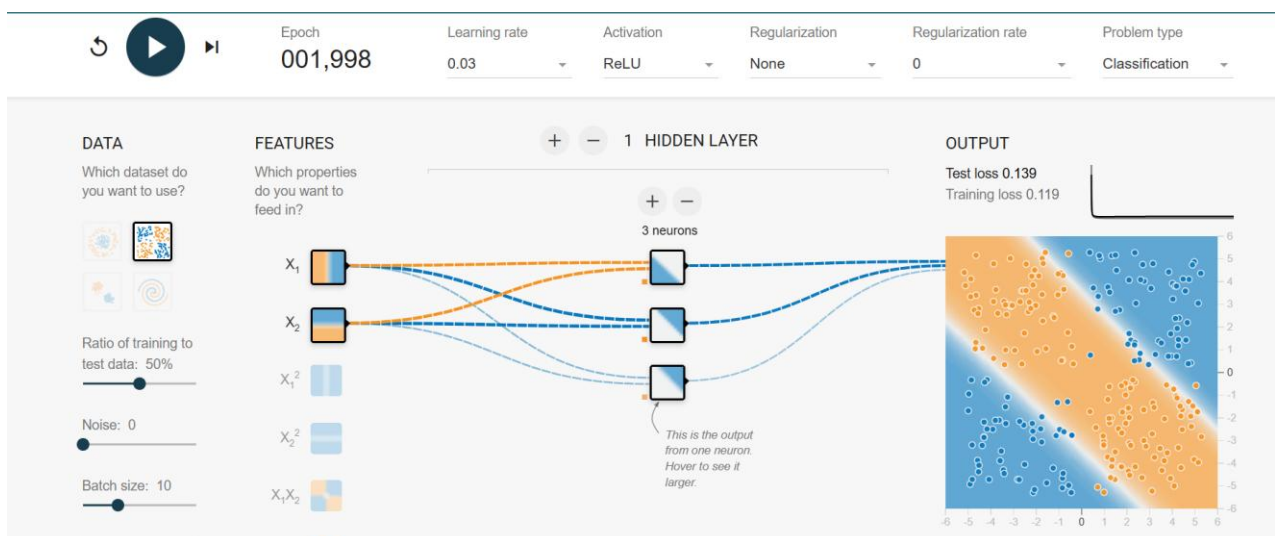
Tensorflow Playground Exercise:

1) What is the smallest network that gives a good fit result?

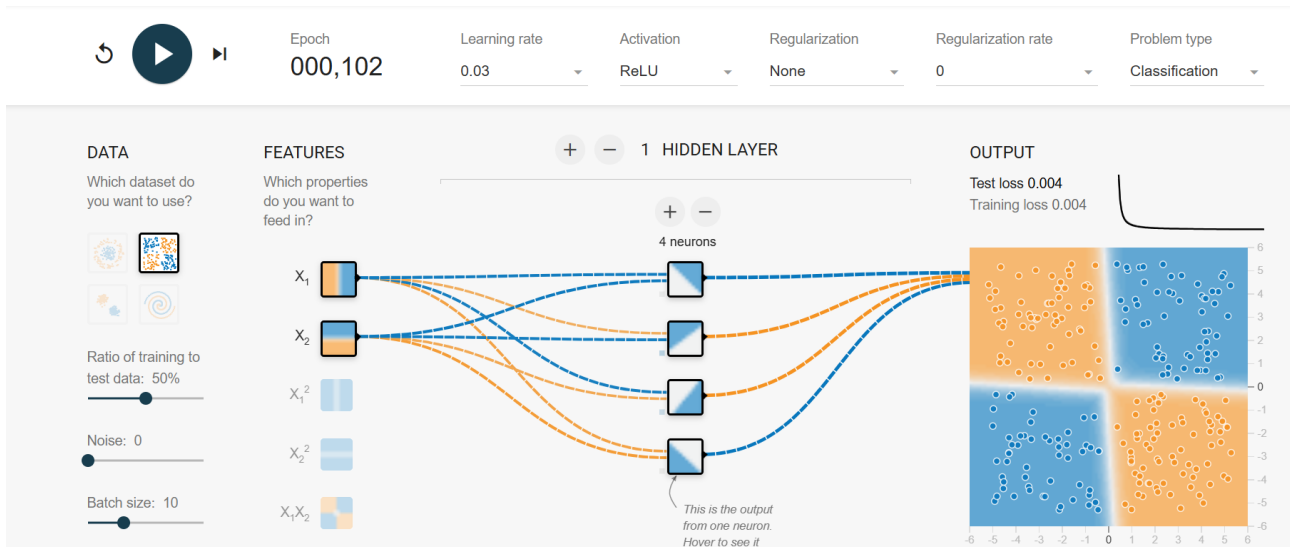
At first I realized that it could be done with 3. As shown in the following image, a network with a single hidden layer and only 3 neurons is mathematically capable of solving the problem. The three neurons combine to create “hourglass shaped” decision boundaries, successfully separating the quadrants, as we can see from the very low loss value for the test dataset. All was done in a reasonable amount of time/epochs.



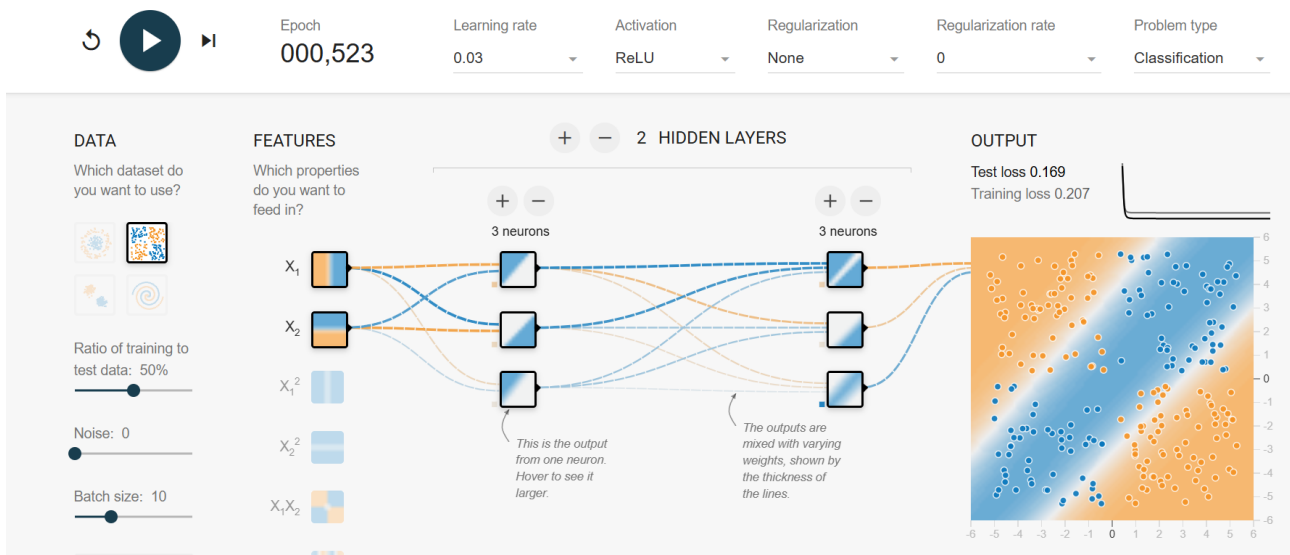
But when I tried again as a confirmation, it didn't always work, even after a lot of epochs. When restarting the training, due to a different random weight initialization, the network struggles to converge or gets completely stuck, failing to perfectly align the three necessary lines.

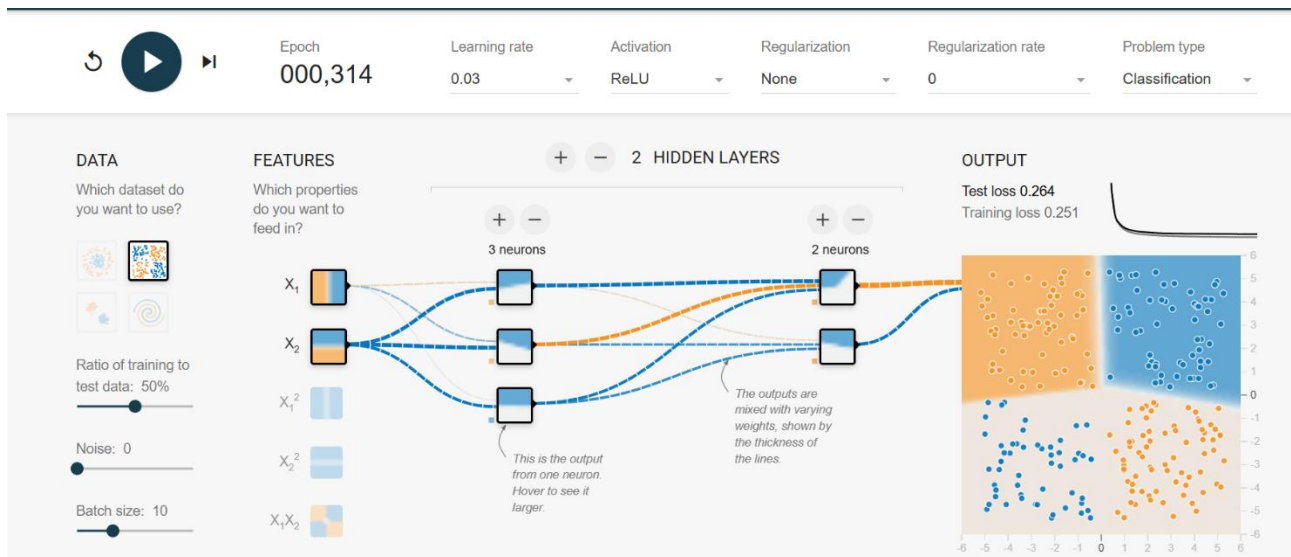


So then I tried with 4 neurons. In this configuration the network reaches stability and perfect discrimination very quickly, as shown in the following image. This demonstrates how adding just one neuron radically changes the situation: the 4 neurons draw 4 straight lines that intersect to form a sort of “X” shape, which isolates the four quadrants in very few epochs.



The following two images show attempts with deeper networks (more hidden layers) while keeping fewer than 4 neurons per layer. We can observe that adding depth does not help if the very first layers do not have enough neurons to make the fundamental spatial 'cuts'.

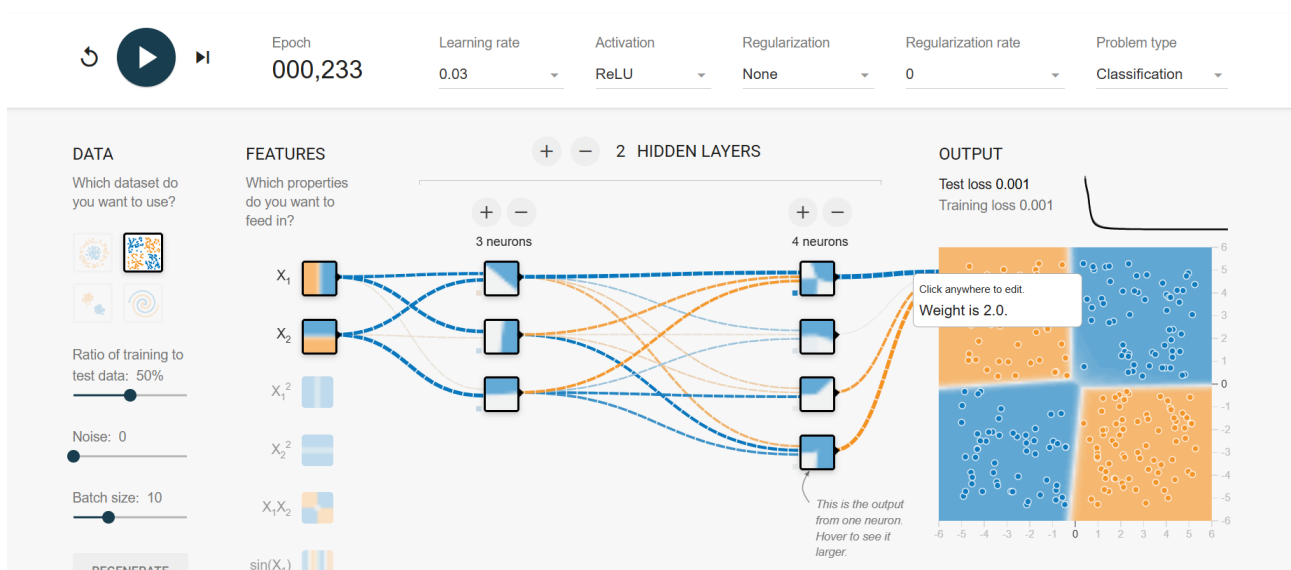


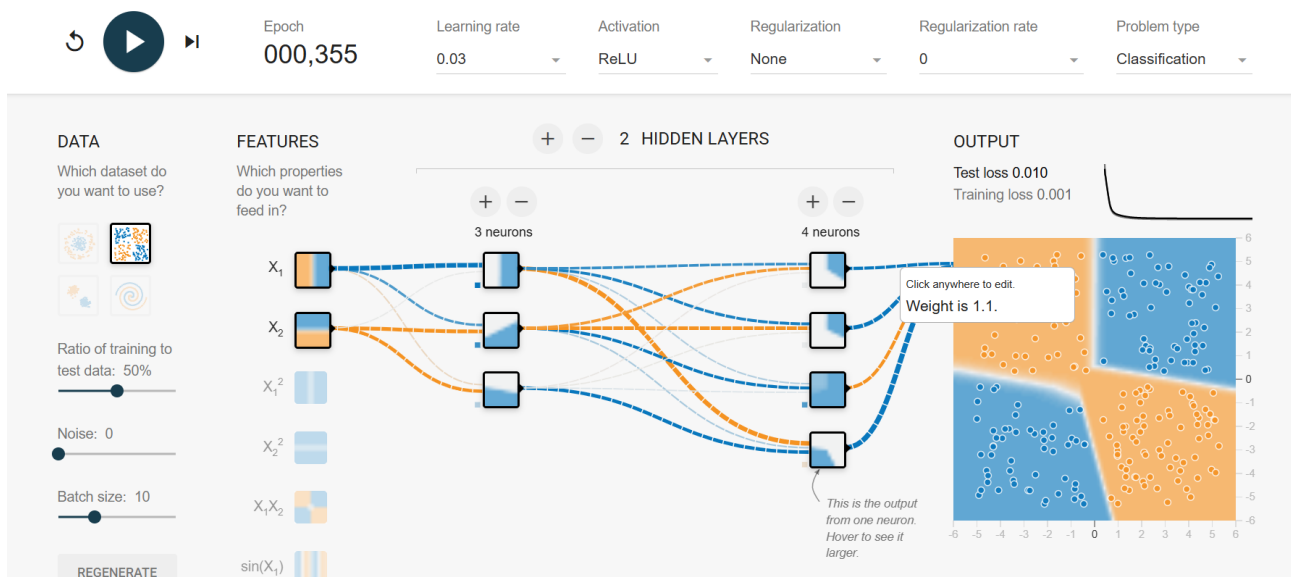


2) What do you observe when training networks with the same settings multiple times?

When restarting the training of the exact same network, as seen before in the minimalist 3-neuron, I observed inconsistent convergence: sometimes the model solves the problem quickly, other times it takes many more epochs, or fails entirely (it gets stuck predicting inaccurately the colours).

Then, by training a more complex network that was able to distinguish the data multiple times, I also observed the appearing of different decision boundaries (as shown in the next two images): the final shape of the decision boundaries looks different every time. This is due to random weight initialization. That's the same reason that caused the 3-neuron network to fail sometimes: if the neurons start in a "bad" position, they might get stuck in a local minimum, and may never find the perfect lines required to separate the checkerboard.

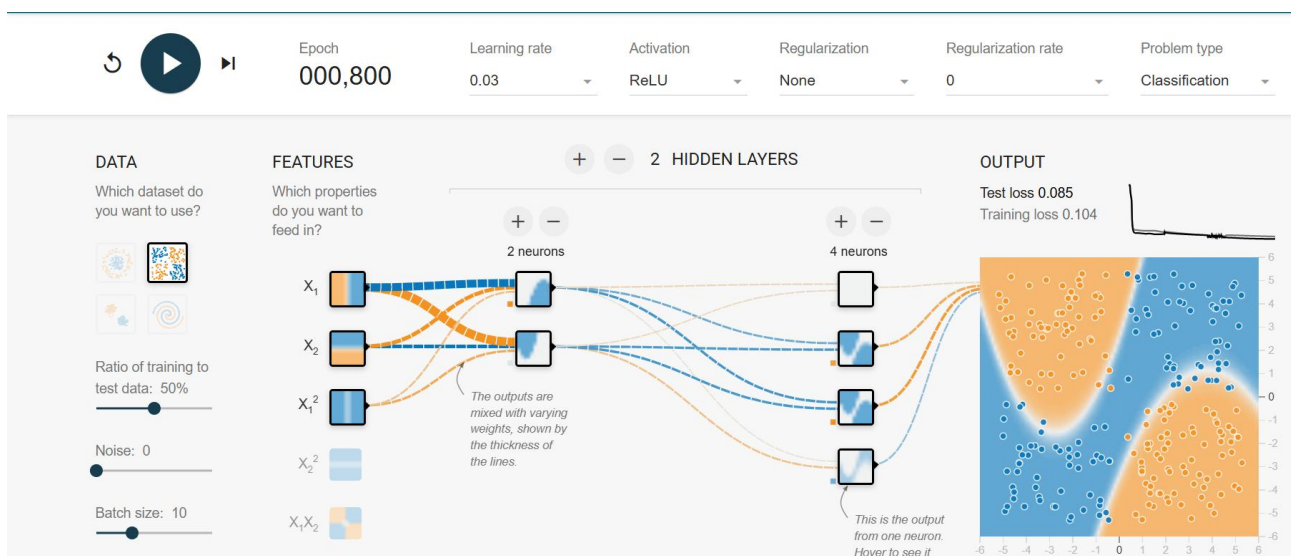




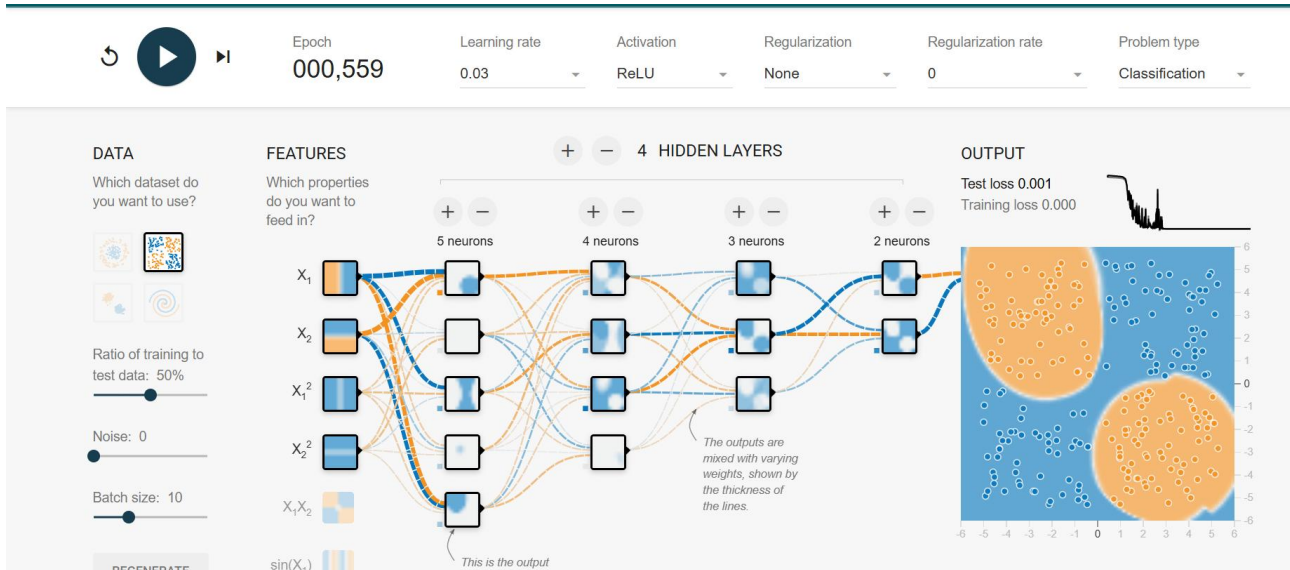
As shown in these images by observing the thickness and colour of the connections between neurons, not only the decision boundaries are different, but also the weights for each neuron (because their output/decision cut changes every training run).

3) Try additional input features: Which one is most helpful?*

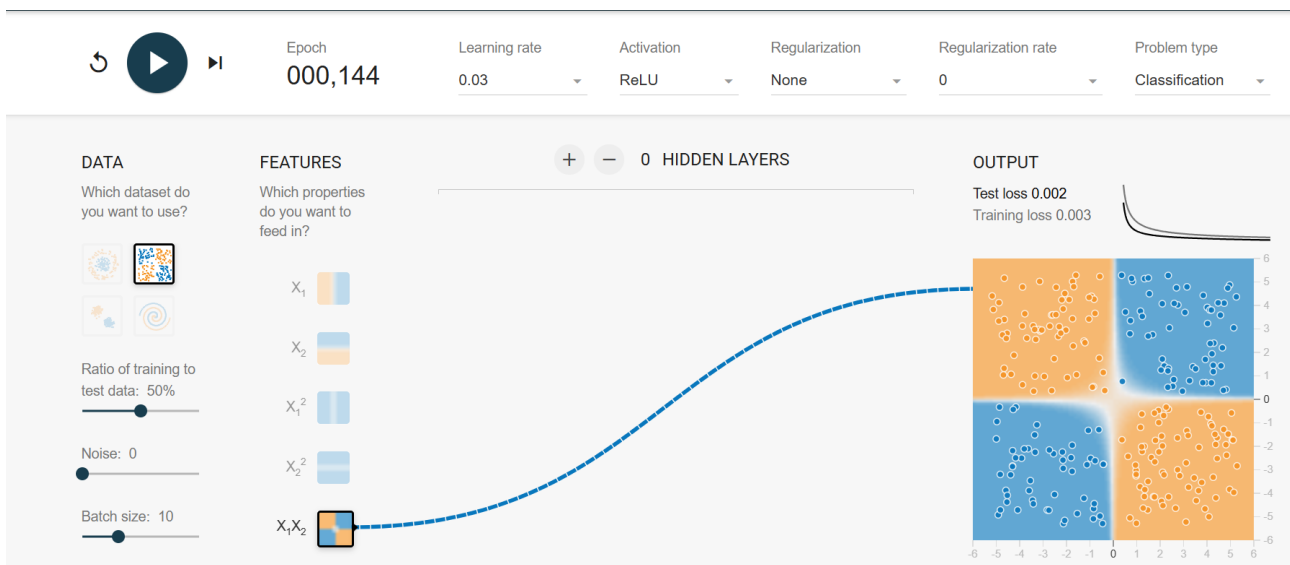
In the following figure, we introduced the quadratic feature x_1^2 . It can be observed that the network tries to exploit this non-linear transformation to create new boundaries, but by itself, it is not decisive enough to solve the checkerboard pattern:



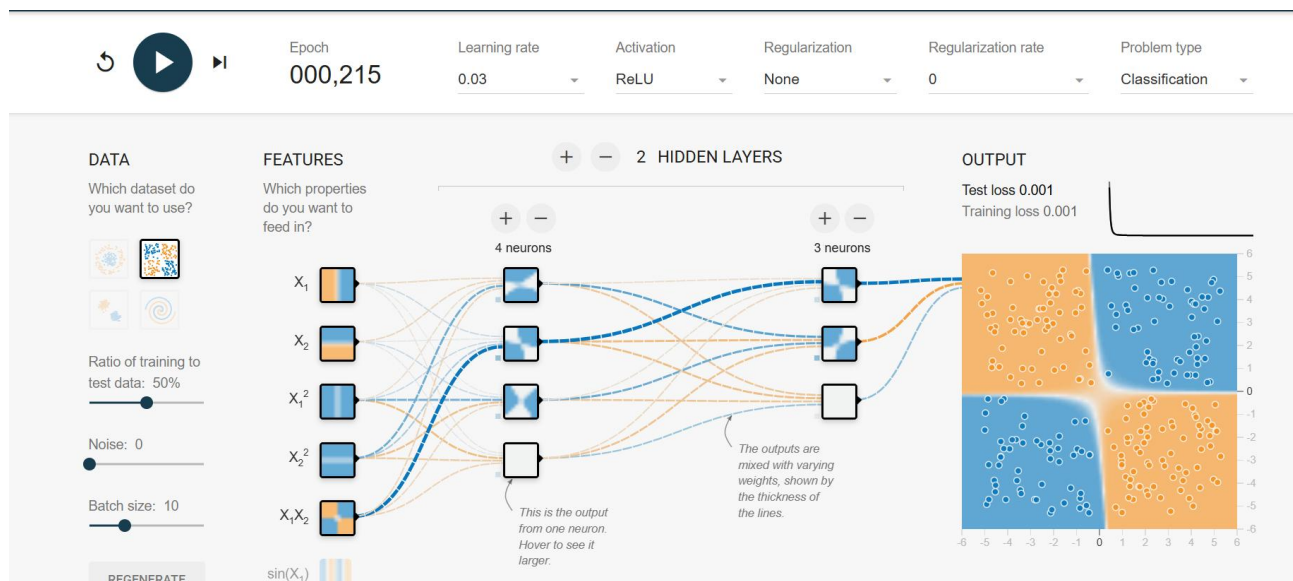
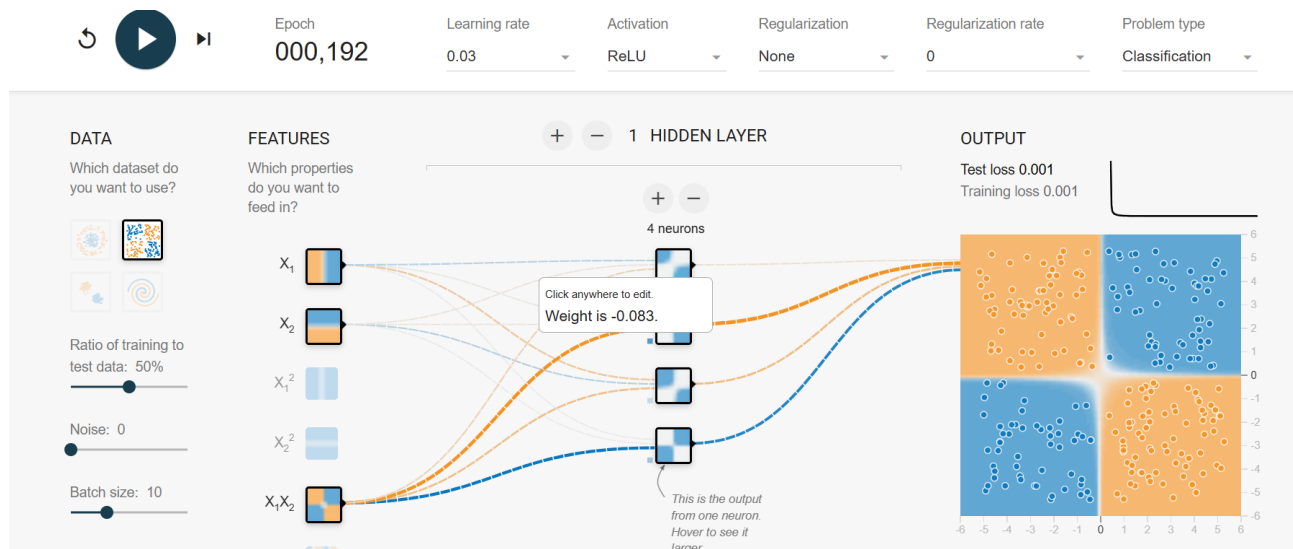
In the next figure I illustrate the addition of the feature x_2^2 . The training is initially unstable, but eventually, the network manages to combine the curves to approximate the boundaries, although it still remains a complex and sub-optimal solution.



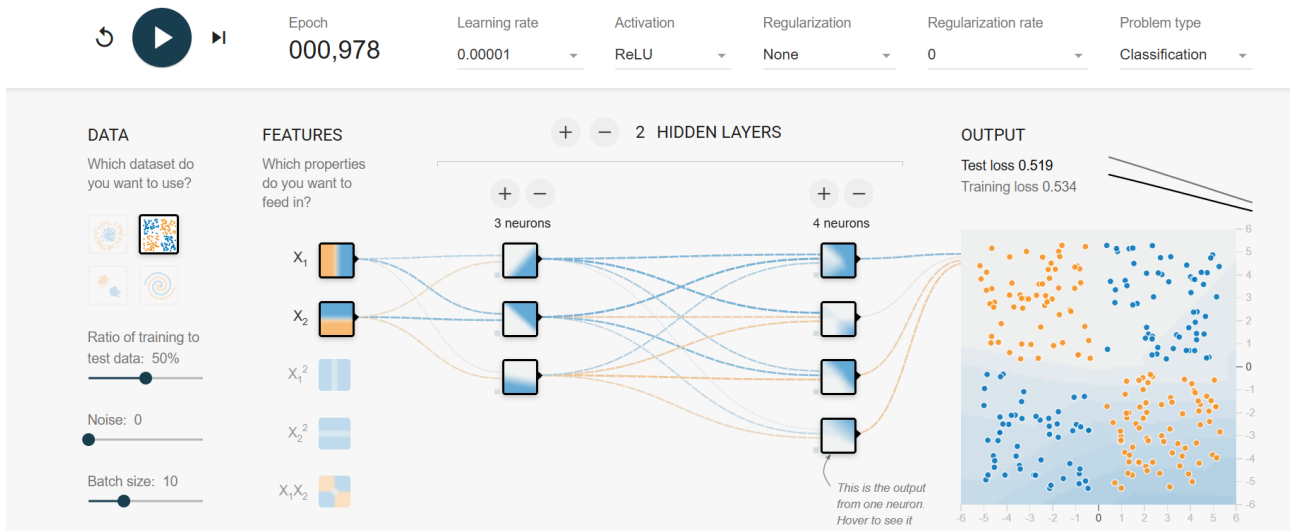
After many tries, the next images represent the perfect and definitive solution. By providing the network with the x_1x_2 feature (the product of the coordinates), the non-linear problem becomes trivially linear. As it can be seen, no hidden layers are necessary: the input connects directly to the output, solving the problem instantly.



Here we can see that even if we are using other features, the x_1x_2 feature does “all the work”.



At the end, I also tried to play by changing other hyperparameters, for example the learning rate. We can see that when it is too small, the algorithm's steps are so tiny that the loss curve drops imperceptibly and the number of epochs required to solve the problem explodes.



Conversely, with a learning rate that is too large, the algorithm becomes unstable, the weights start oscillating chaotically, and the network never manages to find the proper model but stops into a local minimum with a high loss.

